ÉCOLE NATIONALE D'INFORMATIQUE

Master II Professionnel et Ingénieur en Informatique

PROPOSITION DE THÈME DE MÉMOIRE

ET CAHIER DES CHARGES DÉTAILLÉ

Mise en place d'une chaîne DevOps pour le déploiement, l'intégration continue et le monitoring d'une application web de gestion de patients

(Backend Django REST Framework — Frontend Next.js — Base de données PostgreSQL)

Présenté par :

SAMBATRINIAINA Marie Claudia

Sous l'encadrement de : .....

Année universitaire 2025 – 2026

Sommaire

1. Introduction et contexte
2. Thème proposé
3. Objectifs du projet
4. Cahier des charges détaillé
5. Gestion de projet — Méthodologie Agile Scrum avec Notion
6. Outils DevOps gratuits proposés
7. Étapes de réalisation, pas à pas
8. Planning prévisionnel (sprints)
9. Livrables attendus
10. Conclusion

1. Introduction et contexte

Dans le cadre du Mémoire de fin d'études du Master II Informatique, ce document propose un thème de projet centré sur la démarche DevOps appliquée au cycle de vie complet d'une application web métier. Le domaine d'application retenu est la gestion de patients, un domaine riche en règles de gestion, en contraintes de sécurité des données et représentatif des besoins réels des établissements de santé (cliniques, cabinets médicaux, centres de santé).

Le projet ne se limite pas au seul développement fonctionnel de l'application : il vise avant tout à mettre en œuvre, documenter et démontrer une chaîne DevOps complète (versioning, intégration continue, livraison continue, conteneurisation, supervision, sécurité) autour de cette application, ce qui constitue l'apport scientifique et technique attendu d'un mémoire de Master II en informatique.

2. Thème proposé

Titre principal proposé :

« Mise en place d'une chaîne DevOps pour le déploiement, l'intégration continue et le monitoring d'une application web de gestion de patients »

Variantes possibles du titre, au choix de l'étudiante et de l'encadreur :

- « Étude et mise en œuvre d'une chaîne DevOps pour le déploiement continu d'une application de gestion de patients »
- « Conception, développement et industrialisation (DevOps) d'une application web de gestion de patients avec Django, Next.js et PostgreSQL »
- « Automatisation du cycle de vie applicatif — CI/CD, conteneurisation et supervision — appliquée à une application de gestion de patients »

3. Objectifs du projet

3.1 Objectif général

Concevoir, développer et déployer une application web de gestion de patients en appliquant de bout en bout une démarche DevOps outillée, depuis le versioning du code jusqu'à la supervision en production, en s'appuyant exclusivement sur des outils gratuits ou open source.

3.2 Objectifs spécifiques

- Rédiger un cahier des charges complet et une modélisation fonctionnelle du système de gestion de patients ;
- Développer une API REST sécurisée avec Django et Django REST Framework ;
- Développer une interface utilisateur moderne avec Next.js et la bibliothèque de composants shadcn/ui consommant cette API ;
- Modéliser et exploiter une base de données PostgreSQL adaptée au domaine médical ;
- Mettre en place un pipeline d'intégration et de livraison continues (CI/CD) ;
- Conteneuriser l'ensemble de la solution avec Docker ;
- Mettre en place la supervision (monitoring, logs) et un premier niveau de sécurisation de la chaîne ;
- Piloter le projet avec une méthodologie Agile Scrum outillée sur Notion.

4. Cahier des charges détaillé

4.1 Présentation du système

L'application « Gestion de Patients » est un système d'information destiné à un établissement de santé (clinique, cabinet médical ou centre de santé) permettant de centraliser la gestion des dossiers patients, des rendez-vous, des consultations, des prescriptions et de la facturation, tout en assurant la traçabilité et la confidentialité des données médicales.

4.2 Acteurs du système

Acteur	Rôle principal
Administrateur	Gestion des comptes utilisateurs, des rôles, des paramètres généraux de l'application.
Médecin	Consultation des dossiers patients, saisie des consultations, rédaction des prescriptions.
Secrétaire / Réceptionniste	Accueil, création des dossiers patients, gestion des rendez-vous, facturation.
Patient (portail — optionnel V2)	Prise de rendez-vous en ligne, consultation de son historique médical.

4.3 Besoins fonctionnels

L'application est structurée en modules fonctionnels suivants :

N°	Module	Description
1	Authentification & rôles	Connexion sécurisée, gestion des rôles (Admin, Médecin, Secrétaire, Patient), permissions par module.
2	Gestion des patients	Création et mise à jour du dossier patient : identité, contacts, antécédents, allergies, groupe sanguin.
3	Gestion des rendez-vous	Planification, modification, annulation des rendez-vous ; agenda par médecin ; rappels.
4	Gestion des consultations	Saisie du motif, des observations, du diagnostic et du compte rendu de consultation.
5	Prescriptions / ordonnances	Création d'ordonnances (médicament, posologie, durée) et génération d'un document imprimable.
6	Documents médicaux	Dépôt et consultation de pièces jointes (résultats d'analyses, imagerie).
7	Facturation & paiements	Génération de factures liées aux actes, suivi des règlements et des impayés.
8	Tableau de bord & statistiques	Indicateurs clés : nombre de patients, rendez-vous du jour, activité par médecin.
9	Notifications	Rappels de rendez-vous par courriel (et SMS en option).
10	Journal d'audit	Traçabilité des actions sensibles (création, modification, suppression de données médicales).
11	Gestion des stocks (pharmacie)	Suivi des médicaments et fournitures, quantités, seuils et alertes de rupture de stock.
12	Téléconsultation	Mise en relation médecin/patient à distance par visioconférence (module avancé).

N°	Module	Description
13	Export & impression PDF	Génération PDF des dossiers patients, ordonnances et factures, prêts à imprimer.
14	Recherche avancée	Recherche et filtres multicritères sur les patients (nom, date, pathologie, médecin).
15	Plannings & disponibilités	Gestion de l'agenda et des congés/disponibilités de chaque médecin.
16	Multilingue	Interface disponible en français, malgache et anglais.

4.4 Besoins non fonctionnels

Catégorie	Exigence
Sécurité	Mots de passe hachés (Argon2/bcrypt), communications chiffrées HTTPS, contrôle d'accès strict par rôle, protection des données médicales sensibles.
Performance	Temps de réponse de l'API inférieur à 300 ms pour les opérations courantes ; pagination systématique des listes.
Disponibilité	Sauvegardes régulières et automatisées de la base PostgreSQL ; reprise rapide après incident.
Scalabilité	Architecture conteneurisée permettant de dupliquer horizontalement le frontend et le backend.
Portabilité	Déploiement reproductible sur tout environnement disposant de Docker.
Maintenabilité	Code documenté, respect des conventions (PEP 8 côté Django, ESLint/Prettier côté Next.js), tests automatisés.
Ergonomie	Interface responsive, utilisable sur ordinateur et tablette, accessible pour le personnel non technique.

4.5 Architecture technique

Backend : Django + Django REST Framework, exposant une API REST au format JSON ; authentification par jetons (JWT via djangorestframework-simplejwt) ; ORM Django pour l'accès à PostgreSQL.

Frontend : Next.js (React) consommant l'API REST ; rendu hybride (SSR pour les pages publiques, CSR pour l'espace authentifié) ; mise en forme avec Tailwind CSS ; composants d'interface avec shadcn/ui (accessibles, personnalisables, basés sur Radix UI) ; gestion des appels API avec React Query ou Axios.

Base de données : PostgreSQL, hébergeant les entités du domaine (patients, rendez-vous, consultations, prescriptions, factures).

Communication : API REST sécurisée en HTTPS ; possibilité d'évolution vers des notifications temps réel avec Django Channels (WebSocket) en version avancée.

Conteneurisation : Un conteneur Docker par composant (backend, frontend, base de données, reverse proxy), orchestrés avec Docker Compose.

Schéma logique de l'architecture (3 couches) :

- Couche présentation : Next.js (navigateur du personnel médical / du patient)
- Couche application : API REST Django / DRF (règles métier, authentification, validation)
- Couche données : PostgreSQL (persistance), stockage fichiers pour les documents médicaux
- Couche transverse : reverse proxy (Nginx/Traefik) + HTTPS devant l'ensemble

4.6 Modèle conceptuel de données (simplifié)

Entité	Attributs principaux
Utilisateur	id, nom, prénom, courriel, mot de passe, rôle
Patient	id, nom, prénom, date de naissance, contact, antécédents, allergies, groupe sanguin
Médecin	id, utilisateur (lien), spécialité
RendezVous	id, patient (lien), médecin (lien), date/heure, statut
Consultation	id, rendez-vous (lien), motif, observations, diagnostic
Ordonnance	id, consultation (lien), liste de médicaments, posologie, durée
Facture	id, patient (lien), actes, montant, statut de paiement
DocumentMedical	id, patient (lien), fichier, type, date de dépôt
StockMedicament	id, nom, quantité disponible, seuil d'alerte, unité

4.7 Cas d'utilisation principaux (synthèse)

Acteur	Cas d'utilisation
Secrétaire	Créer un dossier patient · Planifier un rendez-vous · Émettre une facture · Gérer le stock de médicaments
Médecin	Consulter un dossier patient · Saisir une consultation · Rédiger une ordonnance · Lancer une téléconsultation
Administrateur	Gérer les comptes utilisateurs · Consulter le journal d'audit · Configurer l'application · Choisir la langue
Patient (V2)	Prendre rendez-vous en ligne · Consulter son historique · Rejoindre une téléconsultation

5. Gestion de projet — Méthodologie Agile Scrum avec Notion

Le projet est piloté selon la méthode Scrum, avec Notion comme unique outil de gestion (backlog, sprints, documentation, journal de bord), ce qui en fait à la fois l'outil de suivi Agile et une partie du livrable documentaire du mémoire.

5.1 Rôles Scrum

- Product Owner : encadreur pédagogique / professionnel du projet
- Scrum Master : l'étudiante, responsable du respect du cadre méthodologique
- Équipe de développement : l'étudiante (développement full-stack et DevOps)

5.2 Organisation des sprints

Durée d'un sprint : 2 semaines.

Nombre de sprints prévus : 8 sprints, soit environ 16 semaines de réalisation.

Cérémonies : Sprint Planning (début de sprint), point quotidien consigné dans le journal de bord (à défaut de mêlée d'équipe réelle), Sprint Review (démonstration en fin de sprint) et Sprint Retrospective (amélioration continue).

5.3 Organisation de l'espace Notion

L'espace de travail Notion du projet est structuré en plusieurs bases et pages :

Page / Base Notion	Contenu
Charte de projet	Objectifs, périmètre, parties prenantes, contraintes.
Product Backlog	Base de données des user stories : titre, description, priorité, story points, statut, sprint associé.
Sprint Board	Vue Kanban filtrée par sprint : À faire / En cours / En revue / Terminé.
Journal de bord	Suivi quotidien de l'avancement, blocages, décisions.
Documentation technique	Architecture, choix techniques, procédures d'installation et de déploiement.
Rétrospectives	Bilan de chaque sprint : points positifs, points à améliorer, actions.

6. Outils DevOps gratuits proposés

L'ensemble des outils proposés ci-dessous sont gratuits (versions communautaires, open source, ou paliers gratuits), ce qui permet de réaliser l'intégralité de la chaîne DevOps sans coût de licence, un point important dans un contexte de mémoire d'étudiant.

Catégorie	Outil proposé	Rôle
Gestion de code source	Git + GitHub (ou GitLab)	Versioning du code, gestion des branches, revue de code.
Conteneurisation	Docker + Docker Compose	Empaqueter et exécuter backend, frontend et base de données de façon isolée et reproductible.
Intégration continue (CI)	GitHub Actions (ou GitLab CI)	Exécution automatique des tests et du linting à chaque envoi de code.
Livraison continue (CD)	GitHub Actions + Docker Hub / GHCR	Construction et publication automatique des images Docker.
Registre d'images	Docker Hub ou GitHub Container Registry	Stockage des images Docker versionnées (paliers gratuits).
Infrastructure as Code	Ansible	Automatiser l'installation et la configuration du serveur de déploiement.
Reverse proxy / HTTPS	Nginx ou Traefik + Let's Encrypt	Exposer l'application en HTTPS avec certificat gratuit.
Supervision (monitoring)	Prometheus + Grafana	Collecte de métriques et tableaux de bord de supervision.
Centralisation des logs	Grafana Loki (léger) ou stack ELK	Consultation centralisée des journaux applicatifs.
Qualité de code	SonarQube Community Edition / SonarCloud	Analyse statique, détection de bugs et de dette technique.
Sécurité des images	Trivy	Scan de vulnérabilités des images Docker.
Tests de sécurité API	OWASP ZAP	Tests de sécurité automatisés sur l'API REST.

Catégorie	Outil proposé	Rôle
Tests de l'API	Postman ou Insomnia	Conception et exécution de collections de tests d'API.
Hébergement de démonstration	Render, Railway ou Fly.io (paliers gratuits)	Déploiement d'une démonstration accessible en ligne.
Gestion de projet Agile	Notion (gratuit pour usage individuel/étudiant)	Backlog, sprints, documentation, journal de bord.
Maquettage UI	Figma (gratuit)	Conception des maquettes d'interface avant développement.
Bibliothèque de composants UI	shadcn/ui (gratuit, open source)	Composants React accessibles et personnalisables (boutons, formulaires, tableaux, modales) pour le frontend Next.js.

7. Étapes de réalisation, pas à pas

Étape 1 — Cadrage et conception (Sprint 0)

- Valider le thème et le cahier des charges avec l'encadreur ;
- Créer l'espace de travail Notion (backlog, sprint board, documentation) ;
- Réaliser les maquettes d'interface avec Figma ;
- Concevoir le modèle de données PostgreSQL (schéma relationnel).

Étape 2 — Mise en place de l'environnement de développement

- Créer le dépôt Git (backend, frontend, infrastructure dans un mono-dépôt ou dépôts séparés) ;
- Rédiger les fichiers Dockerfile pour le backend Django et le frontend Next.js ;
- Écrire un docker-compose.yml de développement (backend, frontend, PostgreSQL, pgAdmin).

Étape 3 — Développement du backend (Django + DRF)

- Modéliser les entités (Patient, Médecin, RendezVous, Consultation, Ordonnance, Facture) ;
- Développer les endpoints REST et l'authentification JWT ;
- Écrire les tests unitaires et d'intégration (pytest-django).

Étape 4 — Développement du frontend (Next.js)

- Installer et configurer Tailwind CSS et shadcn/ui (composants de base : formulaires, tableaux, boutons, modales) ;
- Mettre en place l'authentification côté client et la gestion des rôles ;
- Développer les pages des modules fonctionnels (patients, rendez-vous, consultations, stocks…) avec les composants shadcn/ui ;
- Intégrer les appels à l'API REST et gérer les erreurs et le chargement.

Étape 5 — Intégration continue (CI)

- Configurer GitHub Actions pour exécuter automatiquement le lint et les tests à chaque push ;
- Ajouter la construction automatique des images Docker dans le pipeline.

Étape 6 — Conteneurisation et livraison continue (CD)

- Finaliser les images Docker de production (multi-stage build) ;
- Publier les images sur Docker Hub / GHCR ;
- Automatiser le déploiement vers l'environnement cible (Ansible ou script de déploiement).

Étape 7 — Supervision, logs et sécurité

- Déployer Prometheus et Grafana pour la supervision de l'application et des conteneurs ;
- Centraliser les journaux applicatifs (Grafana Loki) ;
- Scanner les images Docker avec Trivy et exécuter un test de sécurité avec OWASP ZAP.

Étape 8 — Déploiement et mise en production de démonstration

- Déployer l'application sur un hébergement gratuit (Render/Railway/Fly.io) ou une VM dédiée ;
- Configurer le reverse proxy et le certificat HTTPS (Let's Encrypt).

Étape 9 — Documentation et soutenance

- Finaliser la documentation technique et le guide de déploiement ;
- Rédiger le rapport de mémoire à partir du journal de bord Notion ;
- Préparer le support et la démonstration de soutenance.

8. Planning prévisionnel (sprints)

Sprint	Durée	Objectifs principaux
Sprint 0	2 semaines	Cadrage, cahier des charges, maquettes, modèle de données, mise en place Notion.
Sprint 1	2 semaines	Environnement Docker de développement + squelette backend/frontend.
Sprint 2	2 semaines	Authentification, gestion des rôles, module Patients.
Sprint 3	2 semaines	Module Rendez-vous et module Consultations.
Sprint 4	2 semaines	Module Prescriptions/Ordonnances et Documents médicaux.
Sprint 5	2 semaines	Module Facturation, tableau de bord et statistiques.
Sprint 6	2 semaines	Mise en place CI/CD (GitHub Actions) et conteneurisation de production.
Sprint 7	2 semaines	Supervision (Prometheus/Grafana), sécurité (Trivy, OWASP ZAP), déploiement.
Sprint 8	2 semaines	Fonctionnalités avancées : gestion des stocks, téléconsultation, export PDF, recherche avancée, plannings médecins, multilingue.

9. Livrables attendus

- Cahier des charges et dossier de conception validés ;
- Code source versionné (backend Django, frontend Next.js, fichiers d'infrastructure) ;
- Pipeline CI/CD fonctionnel et documenté ;
- Application conteneurisée et déployée (environnement de démonstration) ;

- Tableaux de bord de supervision (Grafana) ;
- Documentation technique complète (architecture, installation, exploitation) ;
- Espace Notion du projet (backlog, sprints, journal de bord, rétrospectives) ;
- Rapport de mémoire et support de soutenance.

10. Conclusion

Ce thème permet à Mademoiselle SAMBATRINIAINA Marie Claudia de traiter, dans le cadre d'un seul projet de fin d'études, à la fois un développement applicatif complet (Django, Next.js, PostgreSQL) et une véritable démarche DevOps outillée de bout en bout, tout en restant réalisable avec des outils exclusivement gratuits et un pilotage Agile rigoureux via Notion. Ce double positionnement — développement et DevOps — constitue une valeur ajoutée forte pour un mémoire de Master II en informatique.